

SQL Injection Security Assessment

ThreadWear Web Application

Vulnerability discovery, controlled exploitation, impact analysis, remediation and retesting

FOCUS

Web Application Security · SQL Injection

ENVIRONMENT

Self-hosted ThreadWear test application

CORE WORK

Reconnaissance · UNION-based SQLi · Privilege impact · Secure coding · Retesting

Prepared by

Ayudh Dahal

Cybersecurity · Networking · Web Application Security

Abstract

SQL Injection is a major web application security vulnerability that occurs when user input is handled unsafely and becomes part of a database query. This report examines SQL Injection from an ethical hacking perspective by discussing its concept, causes, impact, real-world relevance, and mitigation methods. The report also analyses two real incidents, the TalkTalk data breach in 2015 and the Freepik/Flaticon incident in 2020, to show how insecure database interaction can lead to sensitive data exposure, financial penalties, and reputational damage. A practical assessment was carried out on a locally hosted ThreadWear web application in an authorised lab environment. The testing followed penetration testing phases, including planning, reconnaissance, vulnerability identification, exploitation, post-exploitation impact analysis, remediation, and retesting. The findings showed that insecure query construction allowed database information and user account details to be exposed. The vulnerability was later mitigated using prepared statements and parameter binding.

Table of Contents

Acknowledgement.....	Error! Bookmark not defined.
Abstract.....	2
1. Introduction	1
1.1. Ethical Hacking	1
1.2. Overview of SQL Injection	2
1.3. Problem Statement and Statistical Analysis.....	3
1.4. Aim and Objectives	4
1.4.1. Aim	4
1.4.2. Objectives:	4
2. Background	5
2.1. Flow of SQL Injection.....	5
2.2. Types of SQL Injection	7
2.3. Causes of SQL Injection	8
2.4. Mitigation of SQL Injection	9
3. Literature Review	10
3.1. Case Study: TalkTalk Data Breach (2015)	10
4. Methodology and Practical Testing	12
4.1. Planning and Scope.....	12
4.2. Reconnaissance	15
4.3. Vulnerability Identification	16
4.4. Exploitation and Demonstration	18
4.5. Post-Exploitation and Impact Analysis.....	24
Impact Analysis and CVSS Score:.....	28
5. Impact of Electronic Transaction Act 2063 on Ethical Hacking	29

6. Conclusion	30
7. References.....	31
8. Appendix	33
8.1. Background.....	33
8.1.1. Types of SQL Injection.....	33
8.2. Remediation and Retesting.....	35

Table of Figures:

Figure 1: Phases of Ethical Hacking (ACTE, 2016).....	1
Figure 2: SQL Injection (VMWare, 2024)	2
Figure 3: Statistical Analysis	3
Figure 4: Flow of SQL Injection	6
Figure 5: Types of SQL Injection	7
Figure 6: Causes of SQL Injection.....	8
Figure 7: Mitigation of SQL Injection	9
Figure 8: PTES Phases of TalkTalk Data Breach	10
Figure 9: TalkTalk Data Breach 2015 (ICO, 2015)	11
Figure 10: Lessons learned from TalkTalk Data Breach	11
Figure 11: Planning and Scope	12
Figure 12: Local ThreadWear login page showing self-hosted URL.....	13
Figure 13: Normal customer credentials entered on login form.	13
Figure 14: Successful login as a normal customer user.	14
Figure 15: Normal search using keyword 'pants'	15
Figure 16: Normal search using keyword 'cardigans'	15
Figure 17: Payload ' causing SQL syntax error	16
Figure 18: True condition payload returning all product records.	16
Figure 19: False condition payload returning no products.....	17
Figure 20: ORDER BY 5 accepted by the application	18
Figure 21: ORDER BY 6 causing unknown column error.....	18
Figure 22: UNION SELECT NULL,NULL,NULL,NULL,NULL test	19
Figure 23: UNION SELECT 1,2,3,4,5 showing visible output columns.....	19
Figure 24: Current database name revealed using database()	20
Figure 25: Current database user revealed using user()	21
Figure 26: MariaDB version revealed using @@version.....	21
Figure 27: Database names revealed from information_schema.schemata.....	22
Figure 28: Table names revealed from the current database	22
Figure 29: Column names revealed from the users table.....	23
Figure 30: Usernames, passwords, and roles extracted from the users table.	23

Figure 31: Admin credentials entered on login page.	24
Figure 32: Successful access to the ThreadWear admin panel	24
Figure 33: Role dropdown showing customer role changed to admin	25
Figure 34: Ayudh Dahal account successfully promoted to admin	25
Figure 35: Product price modified to 0.00 from admin stock management.....	26
Figure 36: Dashboard showing altered product price as \$0.00	26
Figure 37: Disabling one user's ID	27
Figure 38: User cant login after	27
Figure 39: Impact Analysis	28
Figure 45: CVSS Score	28
Figure 40: Vulnerable query before remediation	35
Figure 41: Secure query after remediation	35
Figure 42: Single quote retest	36
Figure 43: Tautology payload retest.....	36
Figure 44: UNION SELECT retest.....	37

1. Introduction

1.1. Ethical Hacking

Ethical hacking refers to a legal way of investigating the vulnerabilities of the systems, applications, and networks before any hackers attempt to take advantage of those vulnerabilities. Ethical hacking involves conducting tests within a certain scope and following a set procedure. The objective of ethical hacking is to enhance the level of security by finding vulnerabilities, showing what might happen when they are exploited, and coming up with a solution for remediation. The phases of ethical hacking include planning, reconnaissance, vulnerability scanning, exploiting, report writing, and retesting (NIST, 2011).



Figure 1: Phases of Ethical Hacking (ACTE, 2016)

This coursework involves ethical hacking in a lab environment where a vulnerable website is analysed for SQL injection vulnerabilities. This means that the hacking will not target any organization or network but will be conducted purely for educational and learning purposes. It aims at understanding how an attacker may use insecure queries on the database and how the problem may be prevented by using proper coding techniques.

1.2. Overview of SQL Injection

SQL Injection is one of the most well-known and persistent vulnerabilities in web application security. It happens when user input is not appropriately processed, and it is directly sent to a database query, which enables unintended SQL statements to be processed by the back-end system (NIST, 2011) ; (CWE, 2004). Since a lot of websites and applications today utilize databases to store user credentials, personal information, and records of transactions, this vulnerability can have severe outcomes in case secure coding ideas are not implemented. Injection is also still considered as one of the major application security concerns in the most recent OWASP Top 10 where it appears as A05:2025 - Injection (OWASP, 2025).

SQL Injection is mainly caused by poor input validation and the unsafe construction of dynamic SQL queries using direct string concatenation. When an application fails to separate user input from the actual query logic, attackers may be able to read sensitive data, modify database records, bypass authentication mechanisms, or disrupt normal system operations. SQL injection is not only a matter of hacking but also a software and database management concern. The significance of understanding SQL injection in ethical hacking lies in the fact that it shows how faulty programming can lead to exploitable vulnerabilities, as well as the importance of secure coding practices (OWASP, 2016).

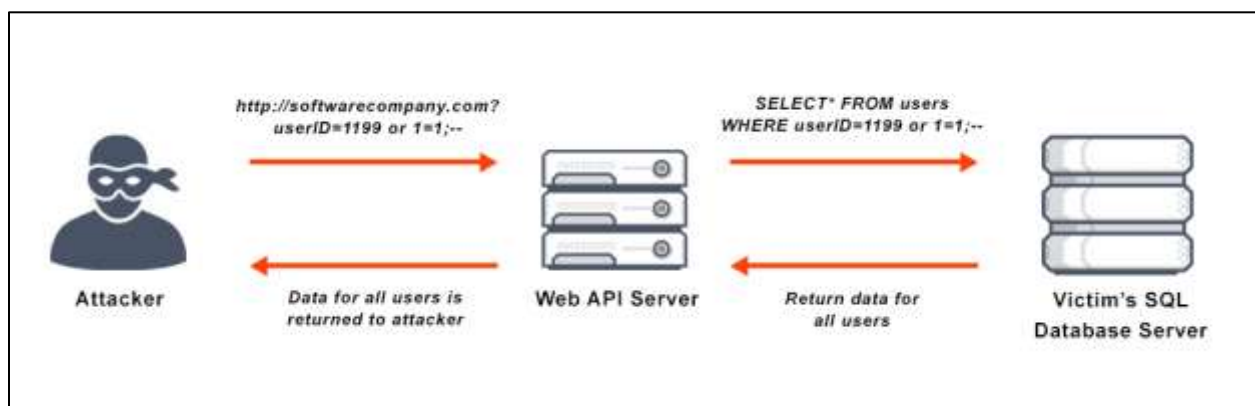


Figure 2: SQL Injection (VMWare, 2024)

1.3. Problem Statement and Statistical Analysis

Despite years of awareness, SQL Injection remains relevant because web applications continue to be heavily targeted in real-world environments. In 2024, Akamai reported over 311 billion web application and API attacks, an increase of 33% over the previous year, indicating that attacks against online systems are not declining but increasing exponentially (Akamai, 2025). Moreover, according to the 2024 report on Data Breaches Investigations, published by Verizon, the number of vulnerabilities exploited as the critical path to launch breaches increased by 180% (Verizon, 2024). These statistics indicate that SQL Injection is not a mere theoretical problem but an ongoing practical menace to confidentiality, integrity, and trust in electronic systems.

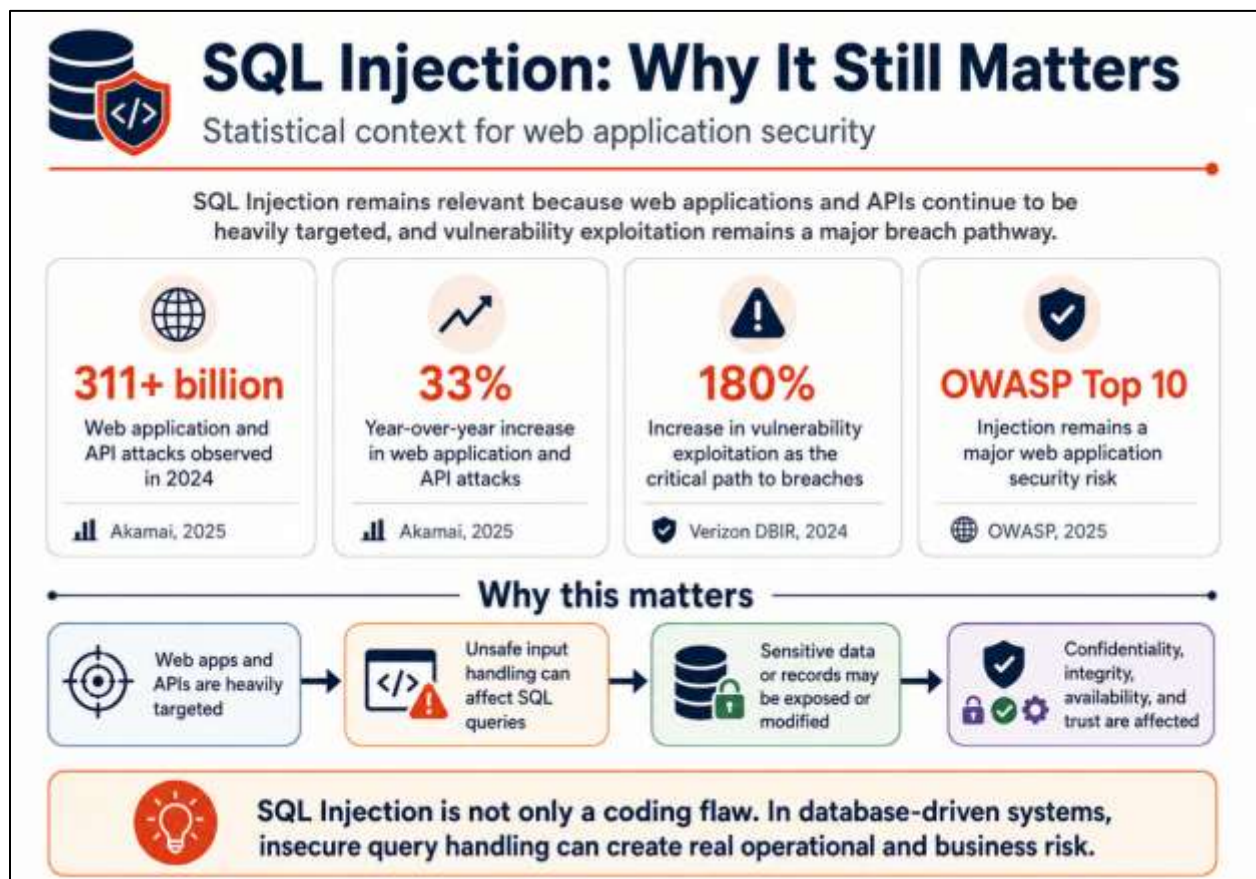


Figure 3: Statistical Analysis

1.4. Aim and Objectives

1.4.1. Aim

To research SQL Injection as an important web application security vulnerability by studying its technical history, practical threat and defensive applicability in ethical hacking.

1.4.2. Objectives:

- To describe the concept and causes of SQL Injection.
- To determine why SQL Injection is still a security issue in web applications today.
- To analyse the effect of SQL Injection on database-based systems.
- To conduct a literature review, case study, and industry evidence on SQL Injection.
- To examine the ethical and legal boundaries of SQL Injection testing by relating the practical investigation to the Electronic Transaction Act 2063 and authorised ethical hacking guidelines.

2. Background

SQL Injection is an application security flaw that arises whenever a user-controllable input is included in a SQL statement without being properly filtered or validated. Database-driven applications have features like login form, search bar, filtering and other URL parameters, which pass user input to the database on the backend (NIST, 2011). Whenever user input is concatenated to form an SQL query, there is a possibility of interpreting the input as a part of the SQL command rather than a regular parameter (OWASP, 2016).

2.1. Flow of SQL Injection

SQL Injection is a multi-step process in which the starts by finding the input source such as search boxes, login forms, filters, or URLs. Normal input is first entered to understand how the application behaves and how it communicates with the backend database. Once that is done, abnormal input is tried out, including input such as quotes, Boolean expressions, SQL comments, ORDER BY statements, and UNION SELECT statements. This helps in understanding whether the input is handled safely by the application, or whether the input itself is used to construct the SQL query. If the application gives out errors from the database or gives out strange outputs or different results depending on whether the condition is true or false, then there may be SQL injection vulnerability (OWASP, 2016).

Once the suspicious behaviour is noticed, the response of the application to each payload is analysed. For instance, if one quotation mark breaks the query, it may imply that the query is not working anymore. If the true statement retrieves data but the false statement does not retrieve any record, then it implies that the database is executing the code. Once this behaviour is confirmed, the tester tries to identify the total number of columns retrieved by the initial query and find out which columns are visible on the web page. This is a demonstration of how a vulnerable input field may expose the behaviour of the database and, in some cases, leak the data that the application should never have revealed (Portswigger, 2020).

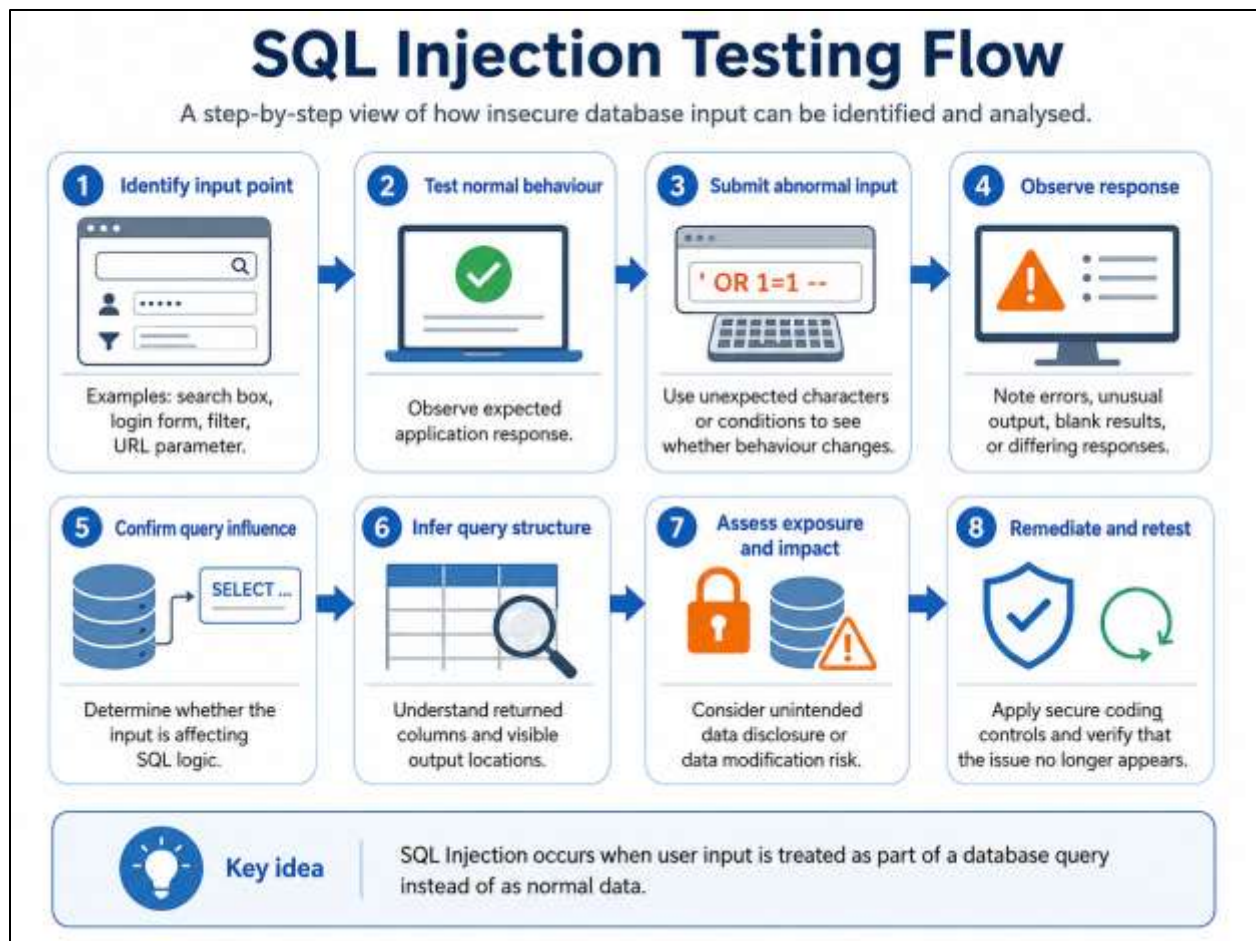


Figure 4: Flow of SQL Injection

2.2. Types of SQL Injection

The main types of SQL Injection are:

1. In-band SQL Injection

- Error-based SQL Injection
- Union-based SQL Injection

2. Blind SQL Injection

- Boolean-based SQL Injection
- Time-based SQL Injection

3. Out-of-band SQL Injection

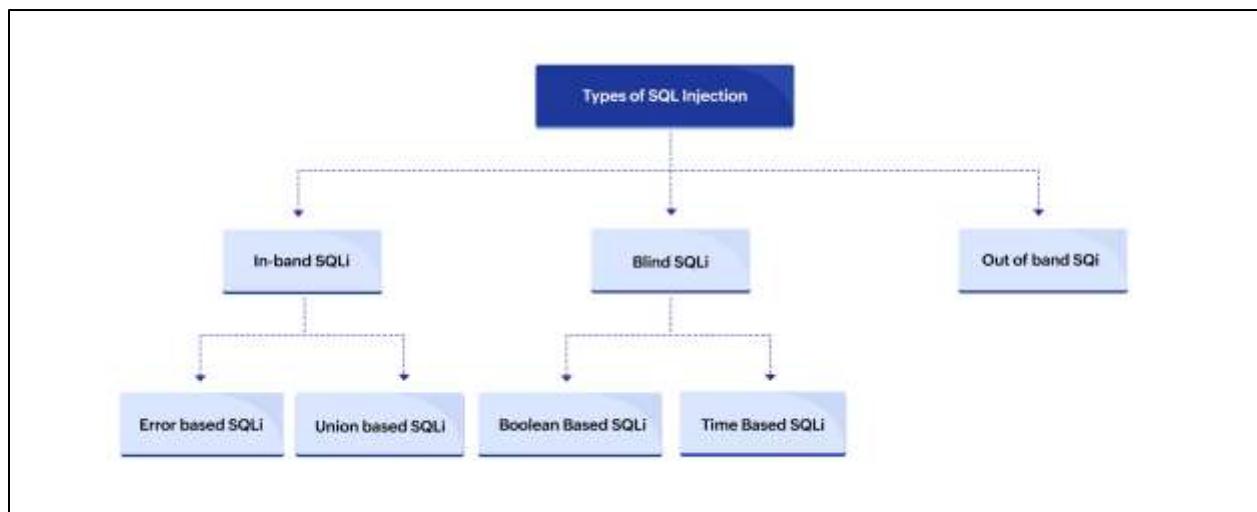


Figure 5: Types of SQL Injection

(For further info about the types of SQL Injection please refer to [Appendix 8.1](#))

2.3. Causes of SQL Injection

SQL Injection commonly occurs when applications build database queries using unsafe user input. This usually happens when developers directly join input values into SQL statements instead of separating data from query logic. Weak input validation, poor error handling, and excessive database privileges can further increase the risk and impact of SQL Injection.

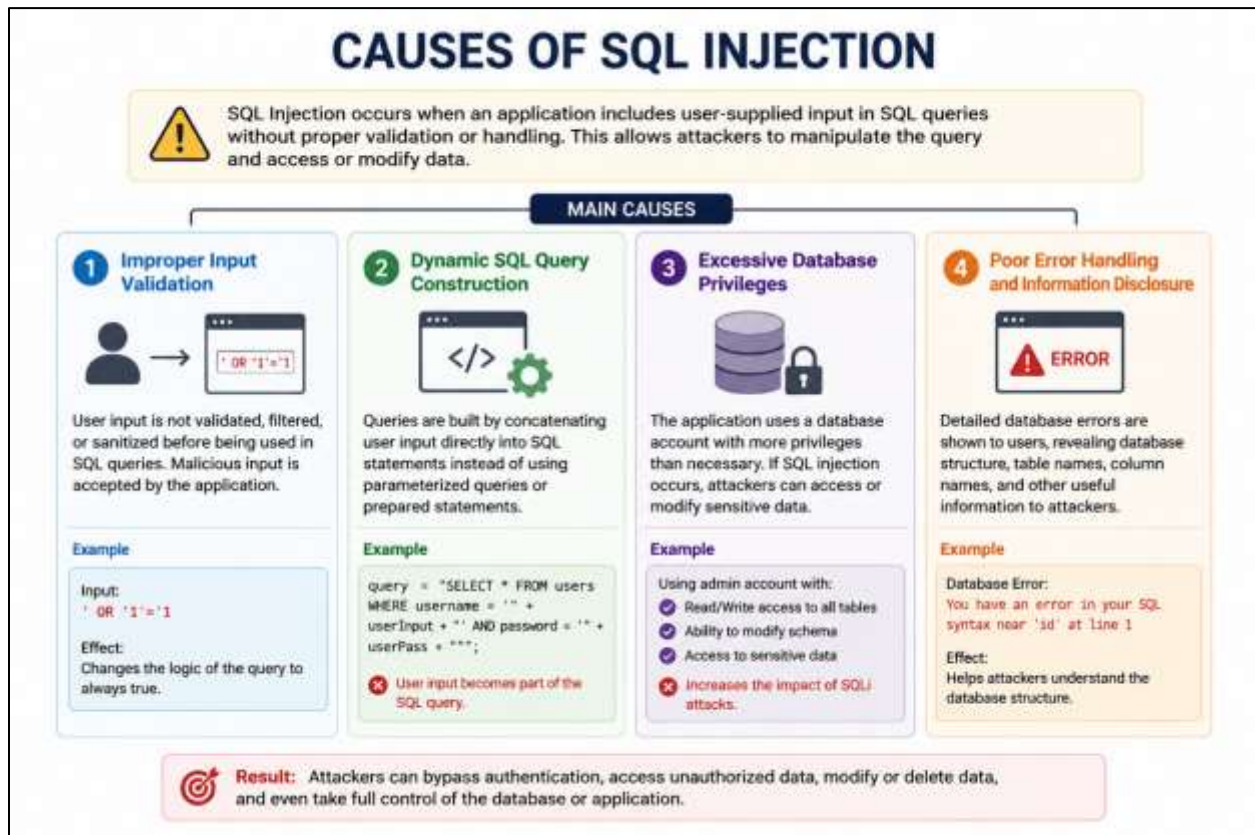


Figure 6: Causes of SQL Injection

2.4. Mitigation of SQL Injection

The main defensive approach against SQL Injection is to ensure that user input is handled safely before it reaches the database. Prepared statements, parameterized queries, input validation, secure error handling, and least-privilege database access are widely recommended practices for reducing SQL Injection risk. These controls help ensure that user input is treated as data rather than executable SQL logic.

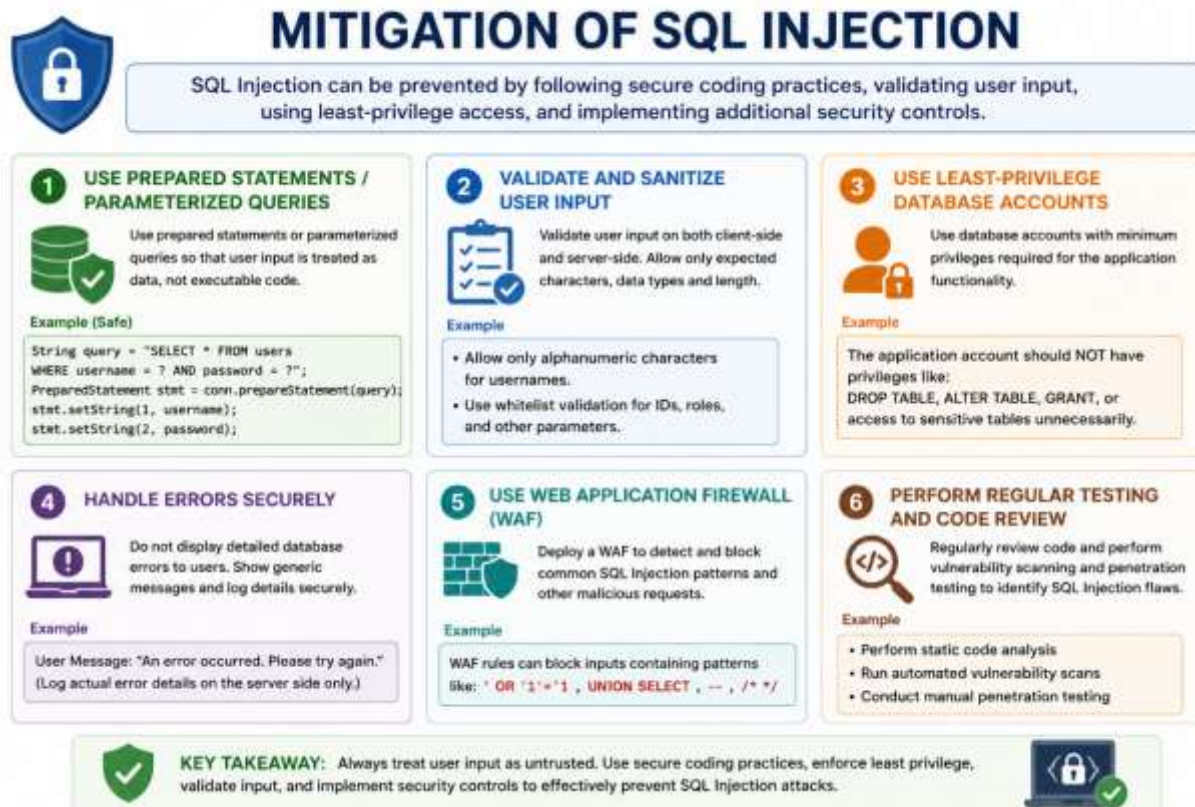


Figure 7: Mitigation of SQL Injection

3. Literature Review

3.1. Case Study: TalkTalk Data Breach (2015)

A relevant case study of a SQL injection attack can be found in the 2015 TalkTalk data breach since it highlights how a simple vulnerability can turn out into a serious security problem if left unattended. From PTES methodology, this incident can be analysed based on reconnaissance, vulnerability identification, exploitation, post-exploitation impact, and remediation failure. As per the Information Commissioner's Office of the UK, the data breach involved an SQL Injection attack on the webpages of TalkTalk, and there existed defences against such an attack at the time (ICO, 2015).

PTES Phase	TalkTalk Case Study Explanation
Reconnaissance	The first stage would involve identifying TalkTalk webpages that accepted user input and communicate with backend databases. In this case, the vulnerable pages were legacy webpages inherited from Tiscali, which remained accessible online and were connected to customer information.
Vulnerability Identification	The attacker would then test whether user-controlled input could affect SQL query behavior. The ICO reported that TalkTalk had already experienced Injection attacks in July 2015, showing that the weakness was known before the October breach.
Exploitation	During exploitation, the SQL Injection vulnerability was used to extract customer records from the database. The breach exposed data belonging to 156,959 customers, including names, addresses, numbers, email addresses, and financial information.
Post-Exploitation Impact	The attack resulted in customer data exposure, reputational harm, financial loss, and regulatory consequences. The ICO fined TalkTalk £400,000 for failing to apply appropriate technical security controls against a preventable SQL Injection attack.

Figure 8: PTES Phases of TalkTalk Data Breach

From an ethical hacking perspective, the most important thing about the post-exploitation impact is that the attack led to information exposure, reputation damage, financial loss and response from the company.



Figure 9: TalkTalk Data Breach 2015 (ICO, 2015)

Lessons Learned:

Area	Lesson Learned from TalkTalk Case
Legacy systems	Old or inherited webpages can remain vulnerable if they are not reviewed and maintained.
Monitoring	Earlier attack attempts must be investigated properly, otherwise the same weakness can be exploited again.
Input handling	User input must not be directly inserted into SQL queries.
Data protection	Customer/account data must be protected from unnecessary exposure.
Remediation	Known vulnerabilities require timely fixes using secure coding controls.

Figure 10: Lessons learned from TalkTalk Data Breach

4. Methodology and Practical Testing

This part discusses the actual SQL injection testing conducted on the locally hosted ThreadWear web application. The testing was done in a safe and controlled environment in the localhost, hence there was no involvement of any other external system or live website during the test. The penetration testing was conducted in an organized manner starting from the planning and scoping phase up to the re-testing phase. In the practical study, the most targeted area was the product search field in which the user input was processed using the q parameter in the URL.

4.1. Planning and Scope

Area	Description
Testing type	Grey-box testing, because one normal customer credential was provided before testing.
Target application	ThreadWear SQL Injection lab hosted locally at localhost/threadwear_sqli_lab.
Access level	Authenticated customer-level access only, not admin access.
In-scope areas	Login page, customer dashboard, product search feature, and admin dashboard access control.
Out-of-scope areas	Any live website, external server, third-party system, or network outside the local lab.
Ethical boundary	Testing was limited to the authorized coursework lab environment only.

Figure 11: Planning and Scope

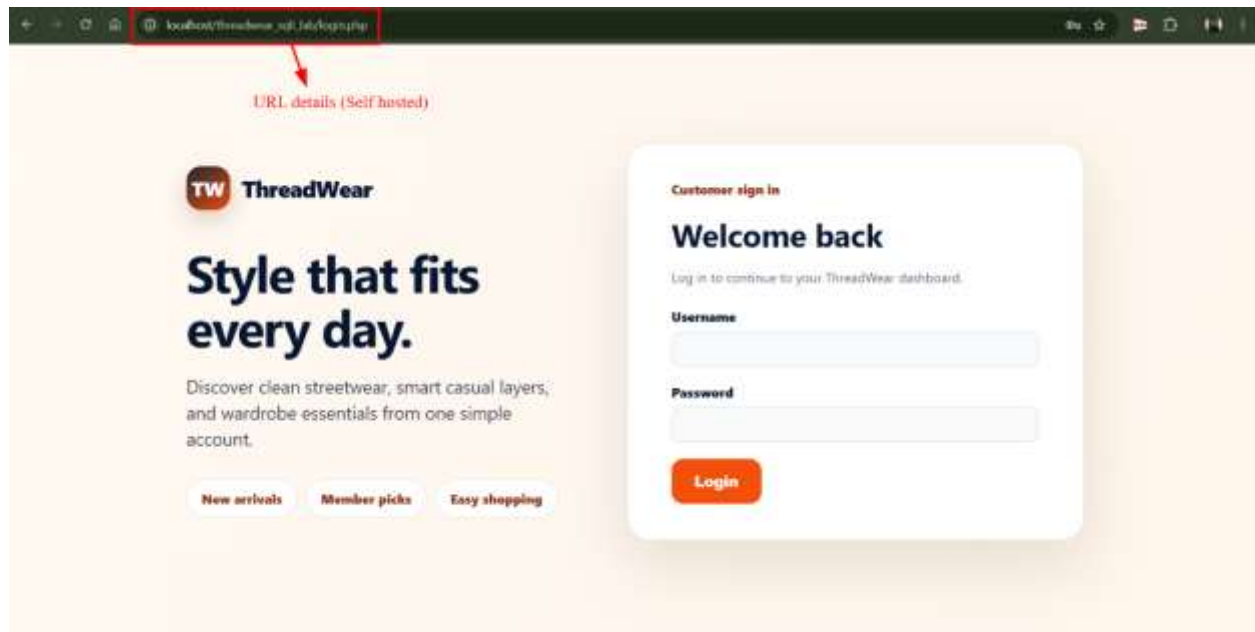


Figure 12: Local ThreadWear login page showing self-hosted URL.

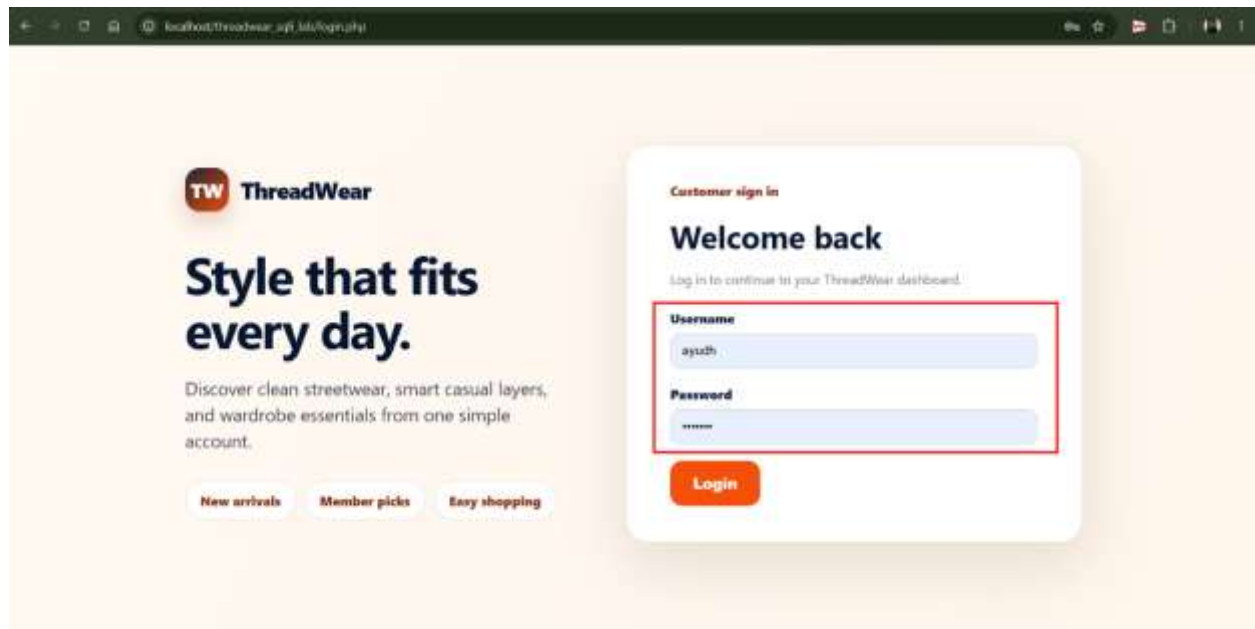


Figure 13: Normal customer credentials entered on login form.

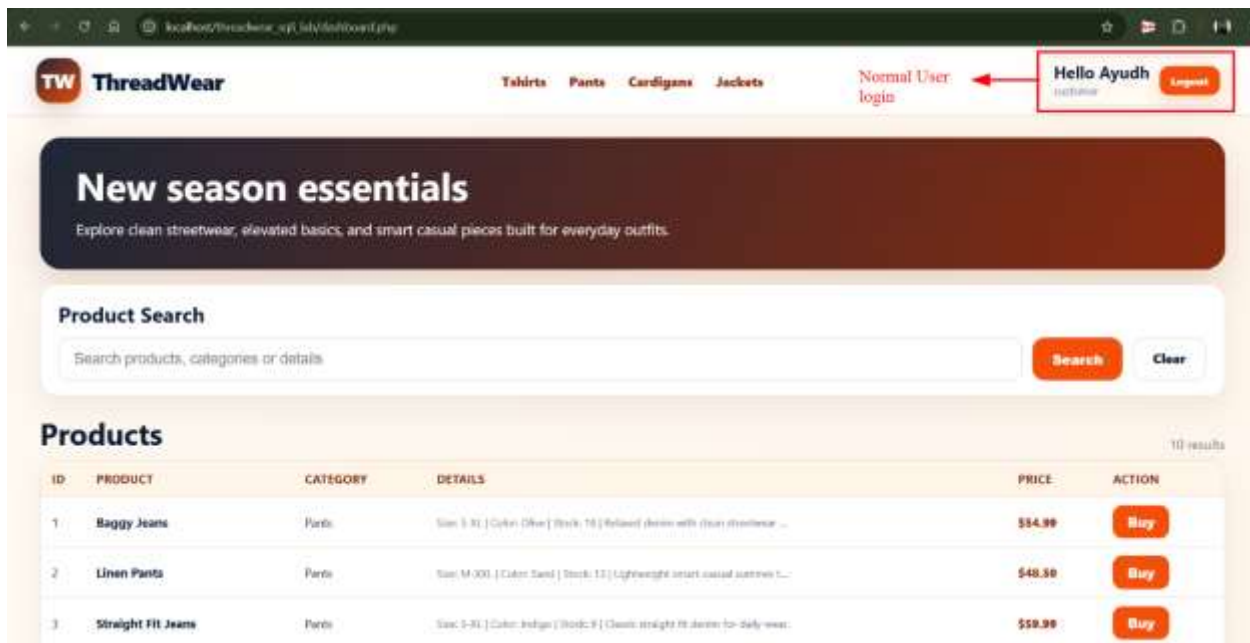


Figure 14: Successful login as a normal customer user.

4.2. Reconnaissance

The reconnaissance process involved monitoring the behavior of the application during normal operation. The search feature was used by making searches using typical keywords such as 'pants' and 'Cardigans'. The searches would append the q parameter to the URL and provide filtered results of products. This indicated that the user inputs provided in the search box were being submitted to the server via a GET method and were being used to query data from the database. Given that the value of the keyword influenced the results, the q parameter was chosen as the primary target for testing.

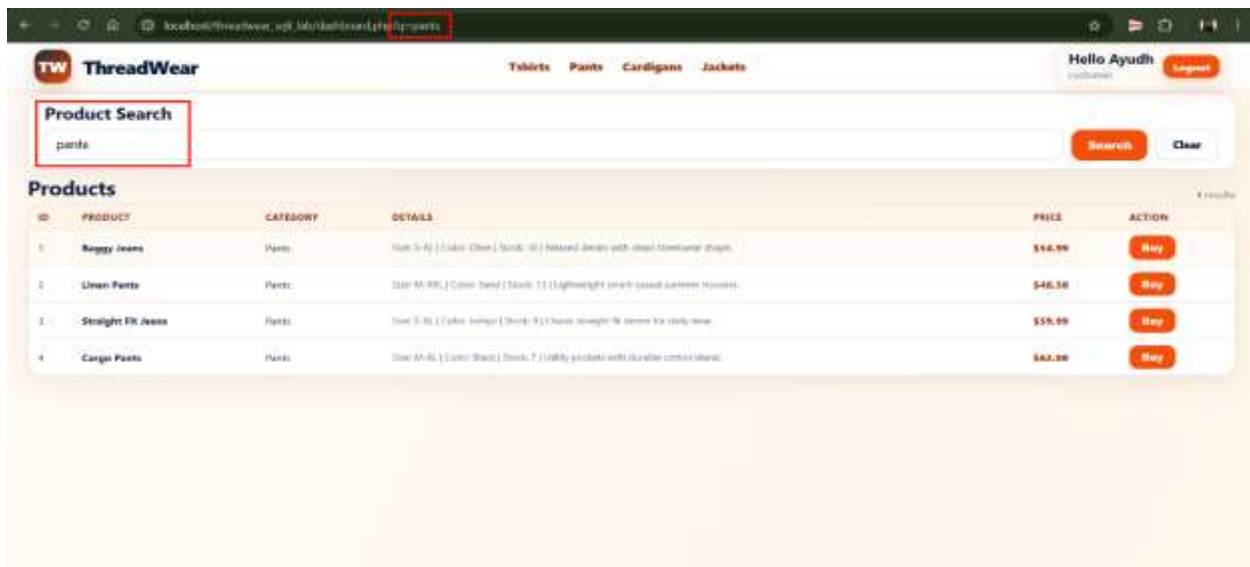


Figure 15: Normal search using keyword 'pants'

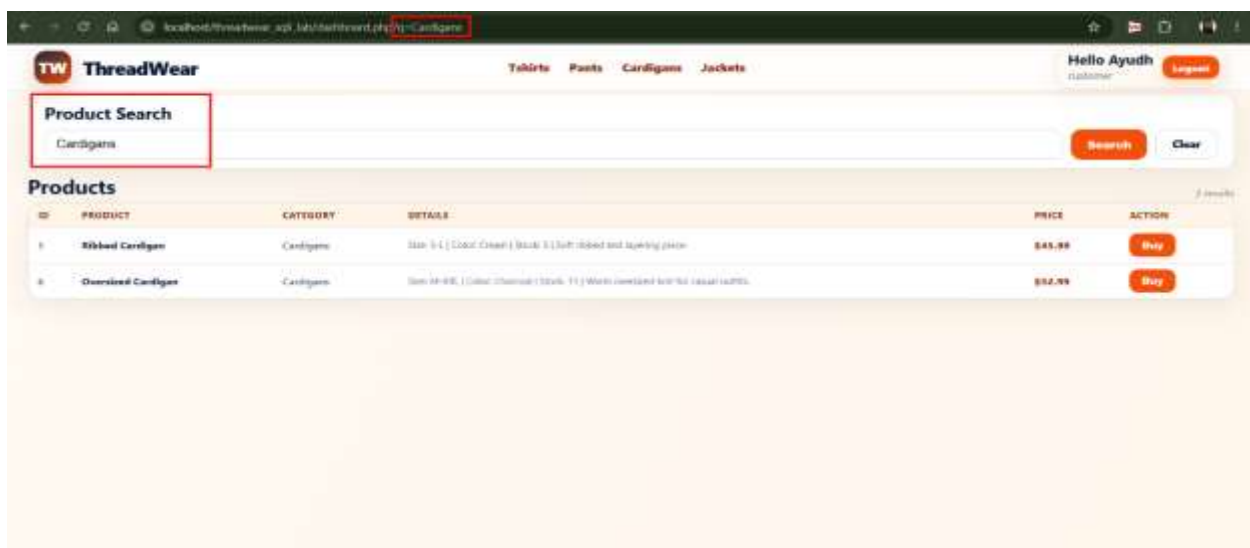


Figure 16: Normal search using keyword 'cardigans'

4.3. Vulnerability Identification

The vulnerability identification phase tested whether the search input could affect the SQL query structure. The initial test submitted was a payload consisting of single quote. The search returned a SQL syntax error, indicating that the input was being successfully inserted into a sql statement without sanitisation. Further Boolean based tests were carried out with true and false conditions. The input '**OR '1'='1**' resulted in all of the product records showing and the input '**AND 1=2--**' returned none. The difference in the results showed that the sql injection was being successful and the search was vulnerable.



Figure 17: Payload ' causing SQL syntax error

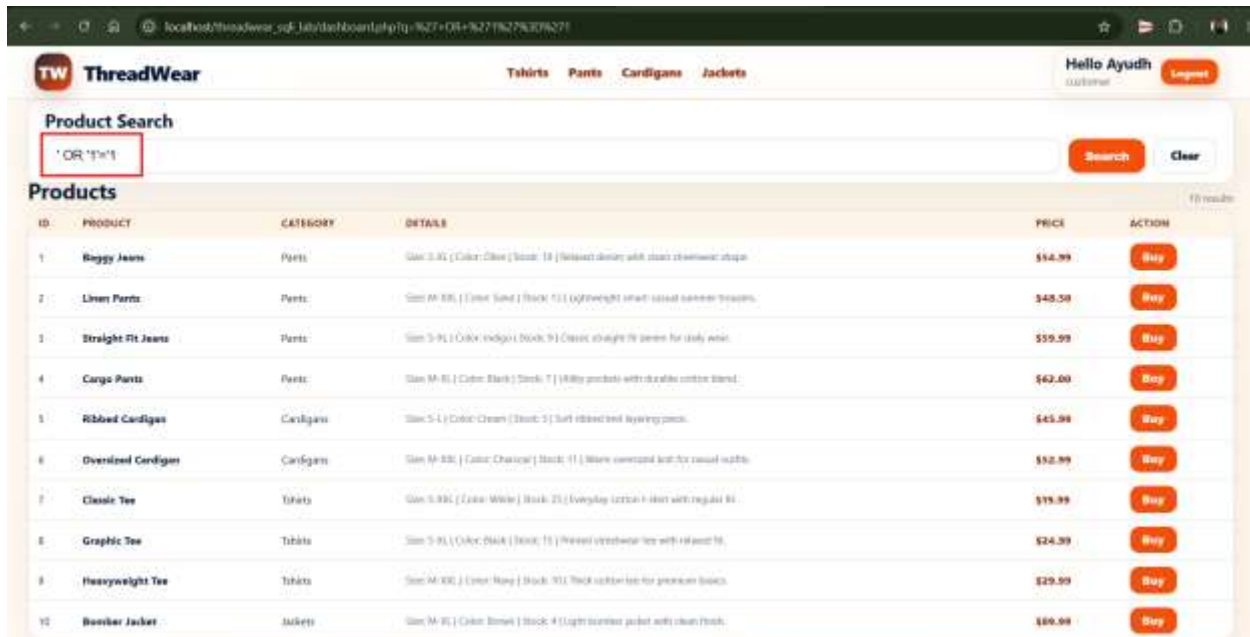


Figure 18: True condition payload returning all product records.



Figure 19: False condition payload returning no products.

4.4. Exploitation and Demonstration

After confirming the vulnerability, exploitation was carried out to determine how much information could be extracted. The initial phase was to find out the number of columns in the initial query. The payload '**ORDER BY 5--**' worked and '**ORDER BY 6--**' returned an 'unknown column' error. This showed that the original query had five columns in it. A UNION SELECT test was then attempted with five columns of data. The numeric payload showed which columns were visible in the page output. This confirmed that injected data could be displayed inside the product table, making the vulnerability suitable for union-based SQL Injection.

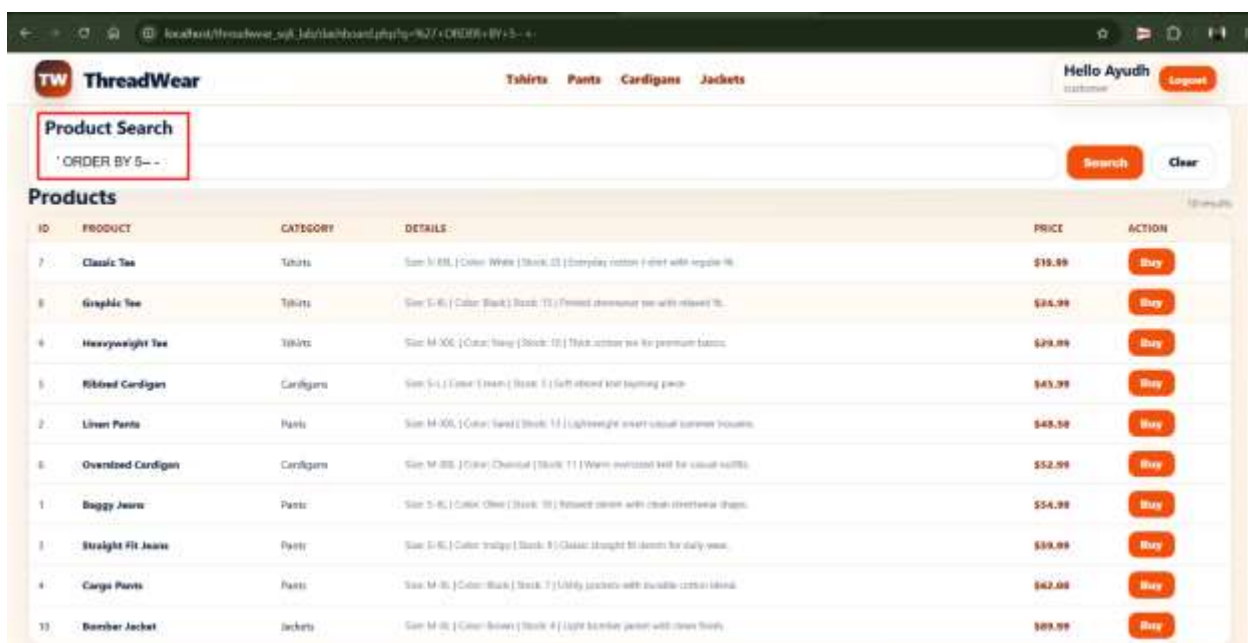


Figure 20: ORDER BY 5 accepted by the application



Figure 21: ORDER BY 6 causing unknown column error

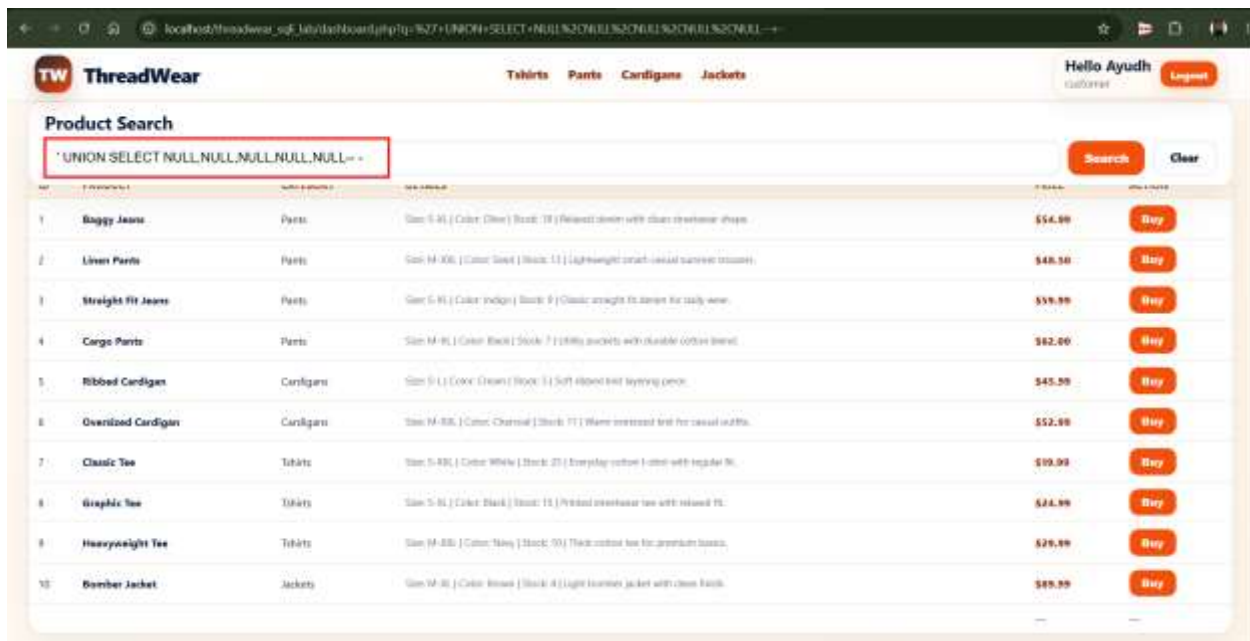


Figure 22: UNION SELECT NULL,NULL,NULL,NULL,NULL test

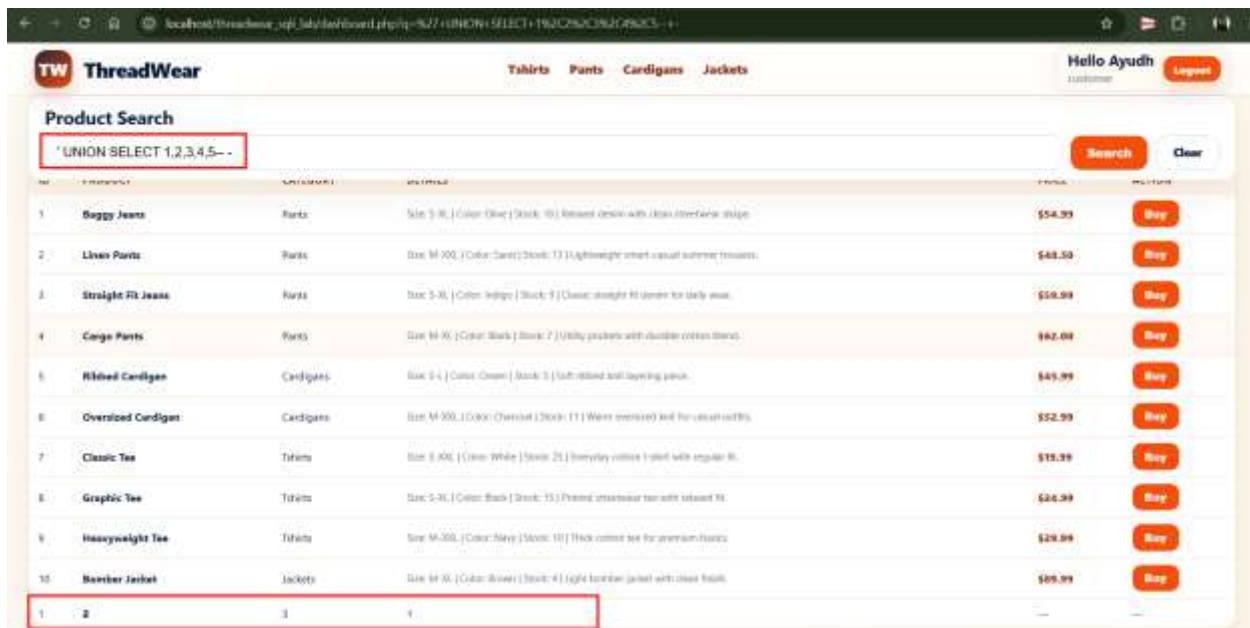


Figure 23: UNION SELECT 1,2,3,4,5 showing visible output columns

After the number of columns and visible output position had been identified, the payloads were built up to return the information stored in the database. Using the **database()** function, the name of the current database was returned, **user()** and **@@version** were then used to determine the current database user and the version of MariaDB. The **information_schema** database was later queried to return the list of databases, the tables in the current database, and the columns in the **users** table. Data was selected from the users table and it returned usernames and password values, plus the roles afforded to each user. This also revealed an **admin account**.

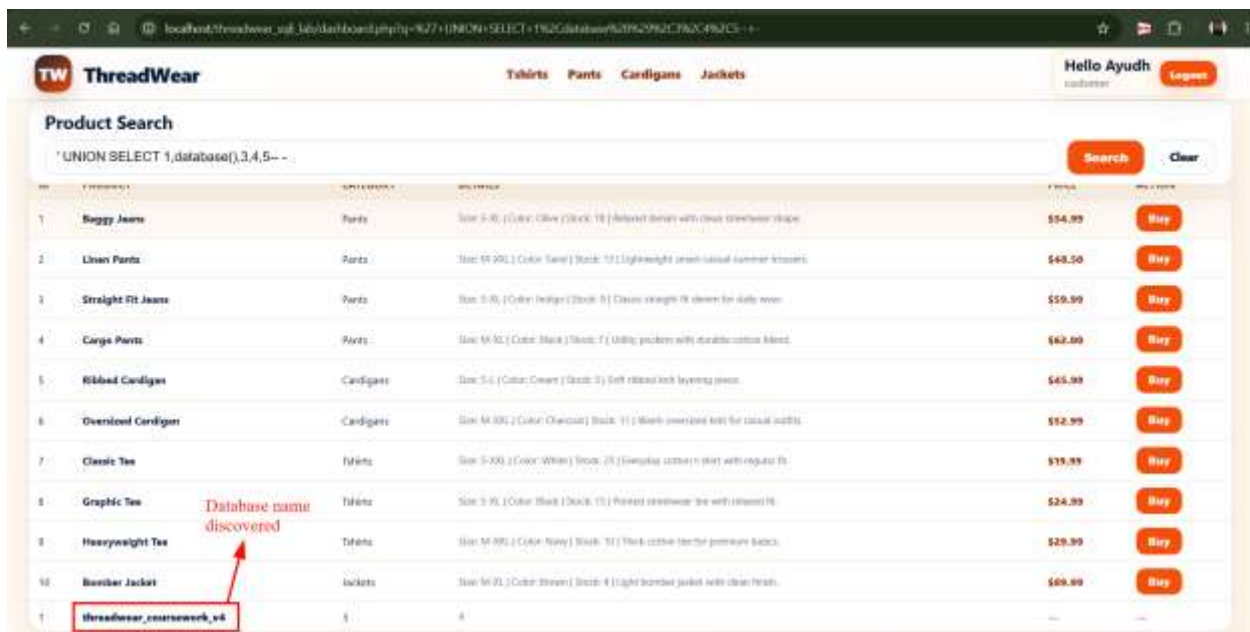


Figure 24: Current database name revealed using database()

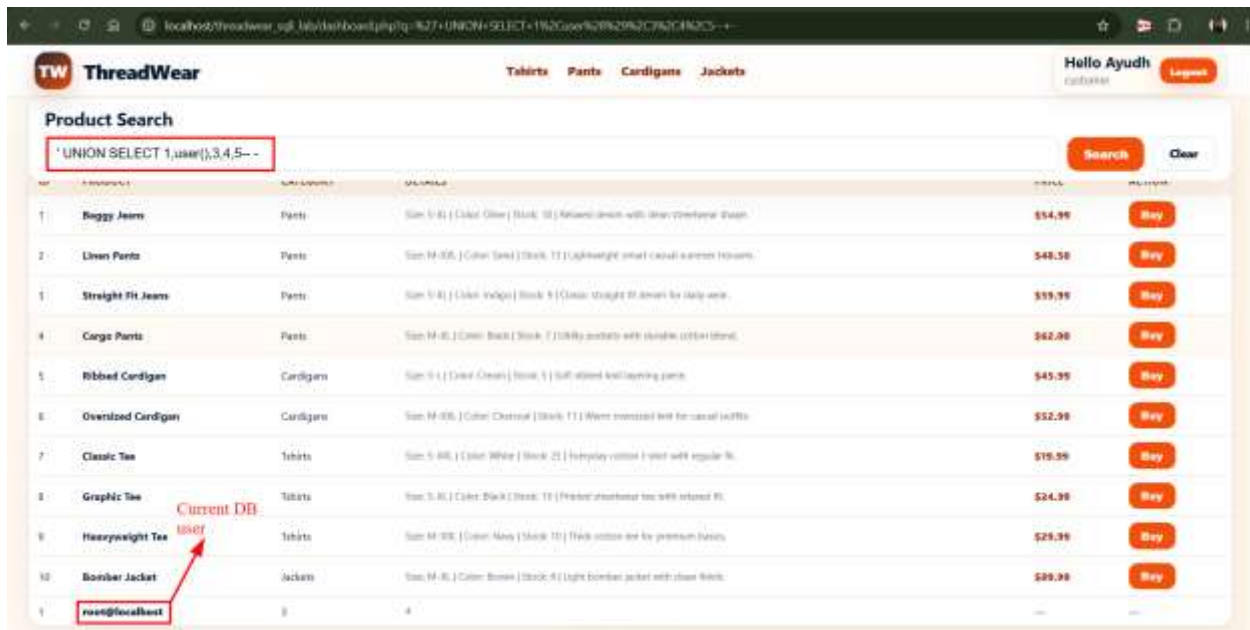


Figure 25: Current database user revealed using user()

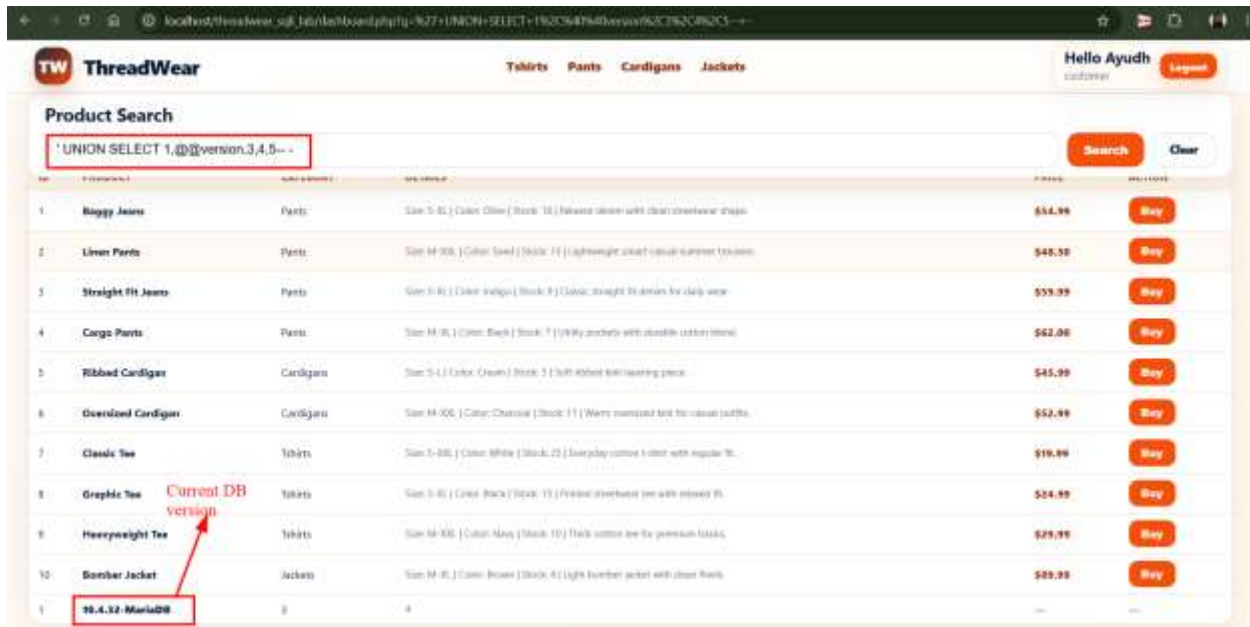


Figure 26: MariaDB version revealed using @@version

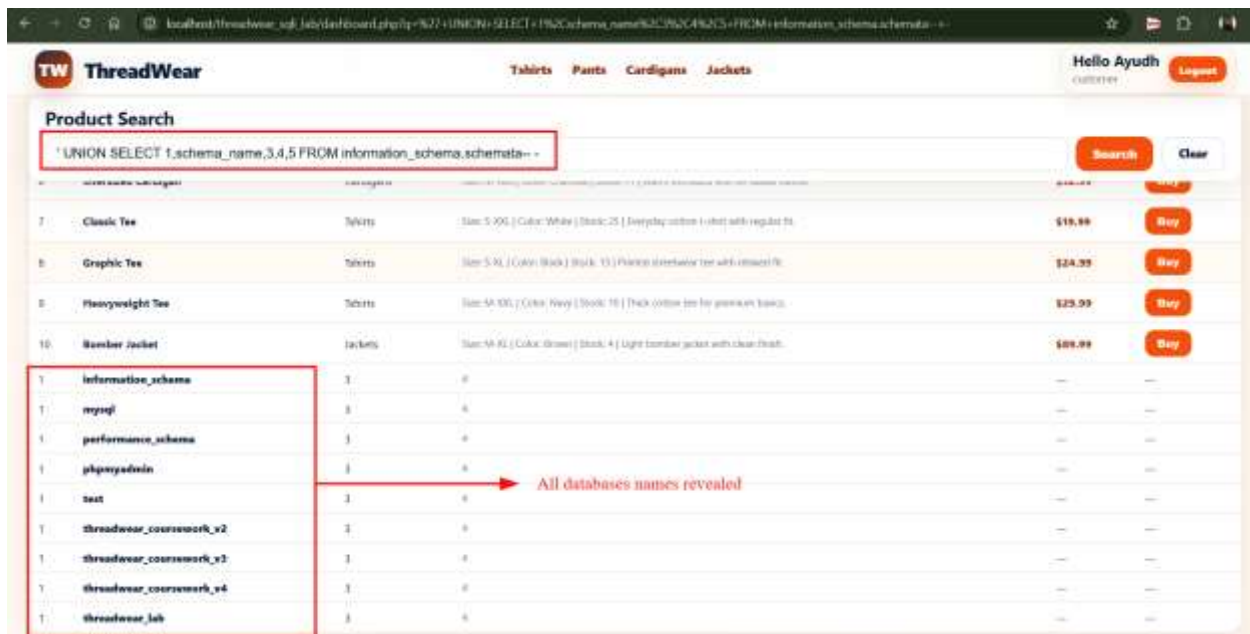


Figure 27: Database names revealed from information_schema.schemata

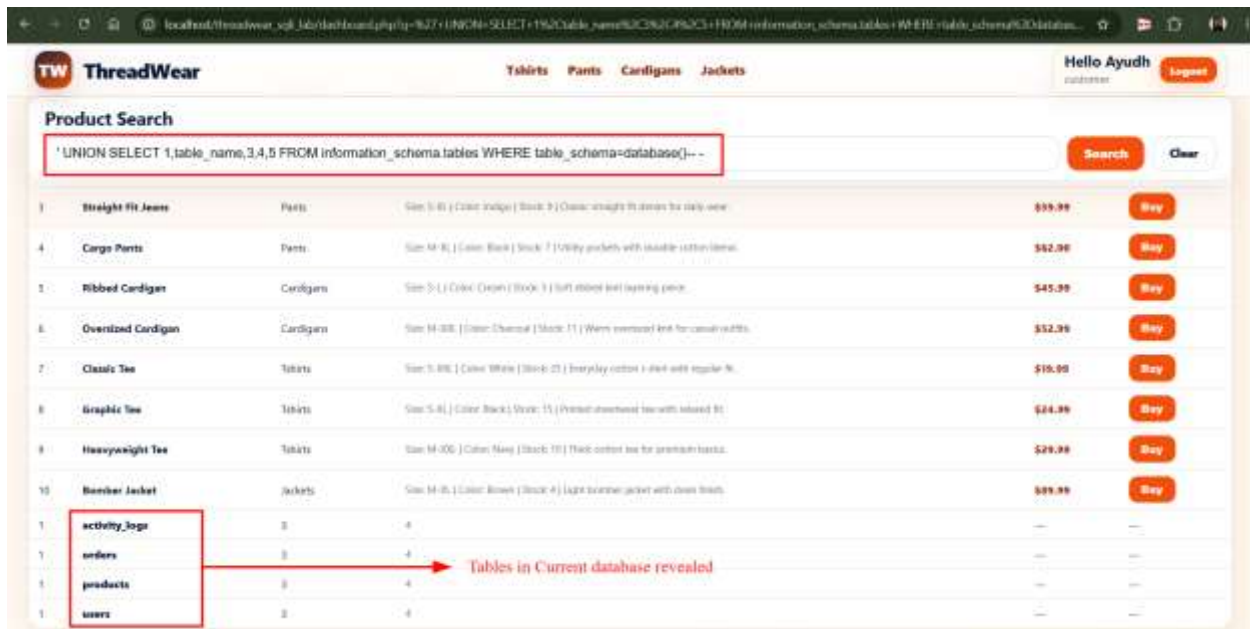


Figure 28: Table names revealed from the current database

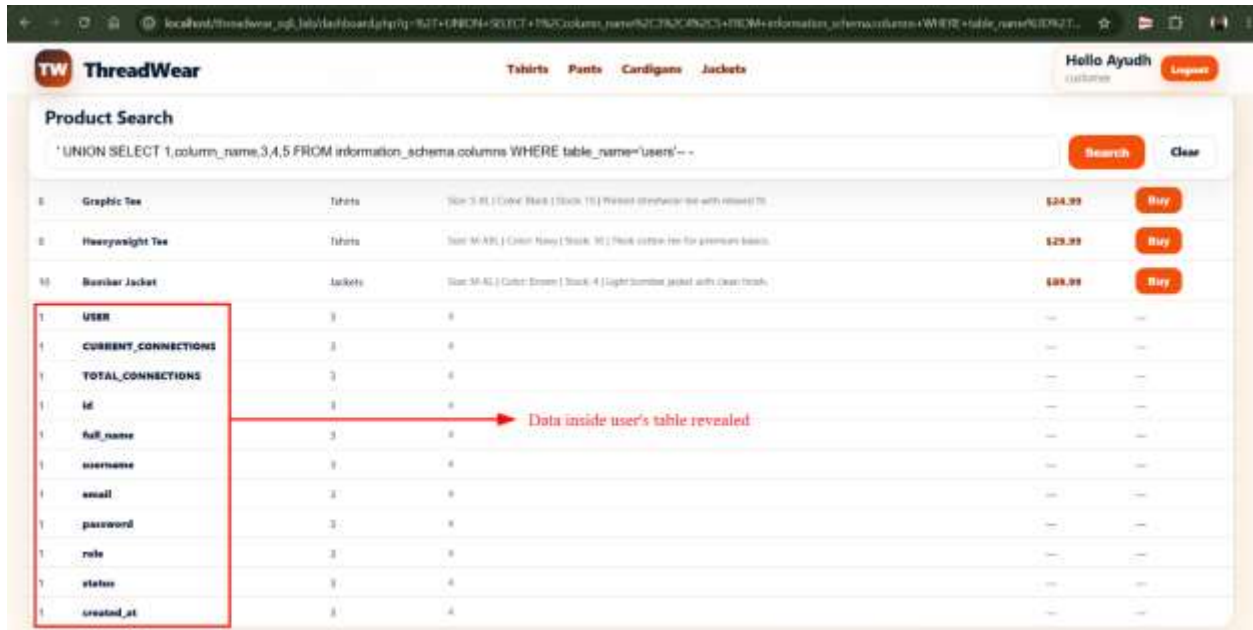


Figure 29: Column names revealed from the users table

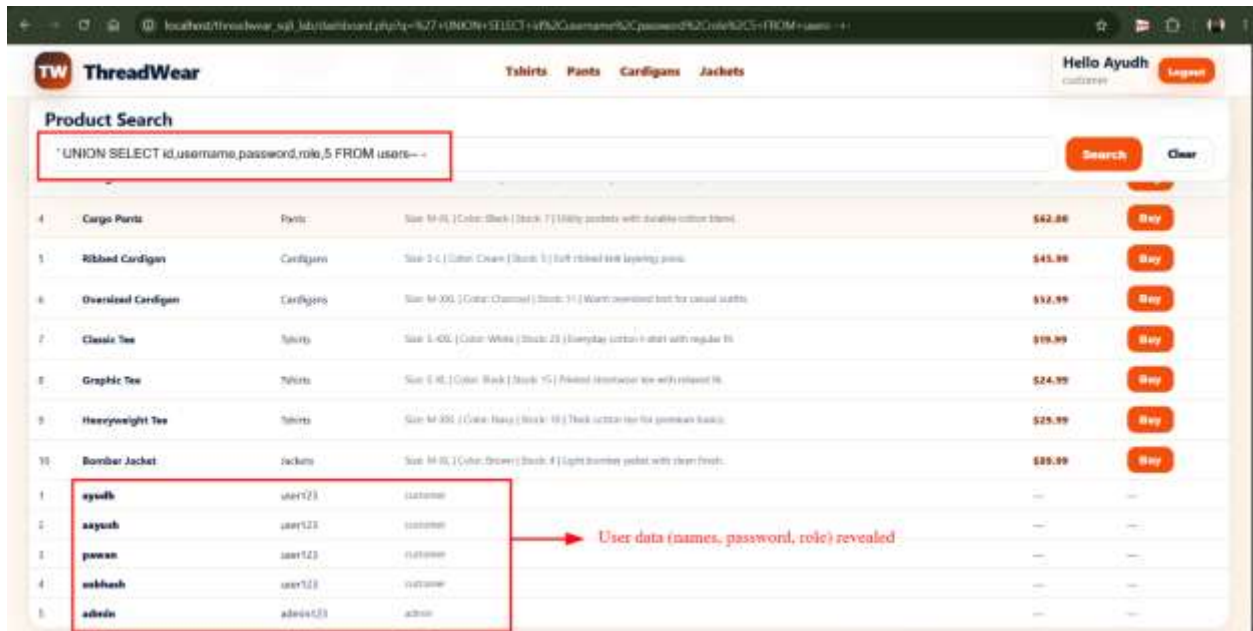


Figure 30: Usernames, passwords, and roles extracted from the users table.

4.5. Post-Exploitation and Impact Analysis

Using the discovered administrator credentials, access was gained to the admin panel, which demonstrated a serious compromise of confidentiality and access control. Once inside the admin area, sensitive management functions became available, including user role modification, stock/product control, and account status changes. As evidence, the compromised access allowed a normal customer account to be promoted to admin, a product price to be changed to zero, and another user account to be disabled.

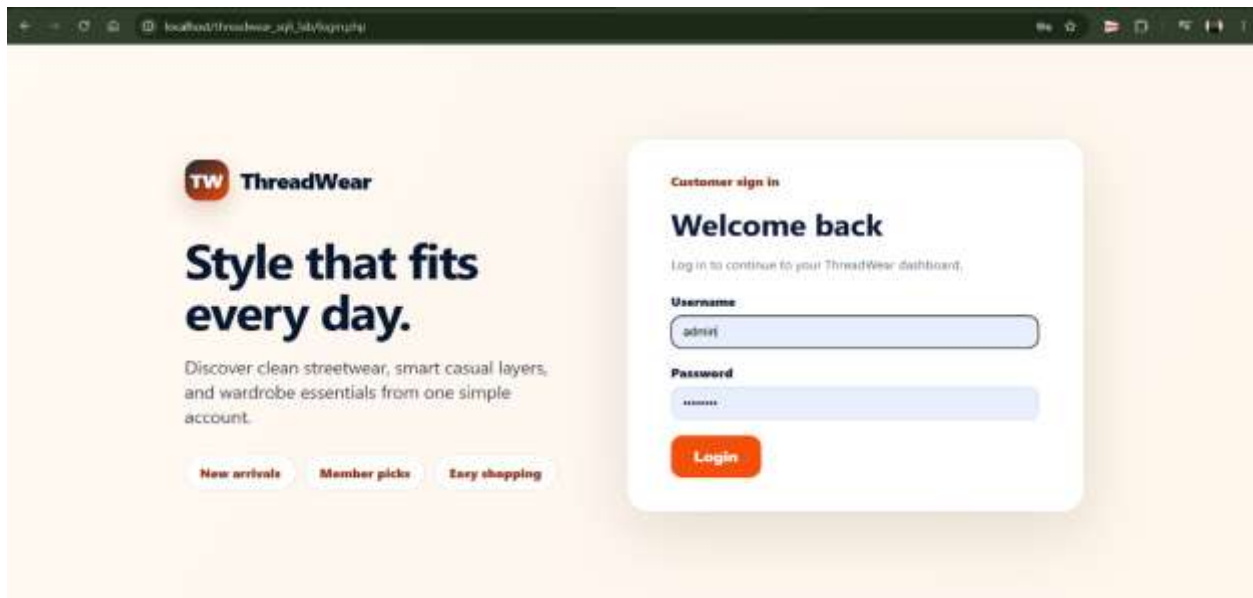


Figure 31: Admin credentials entered on login page.

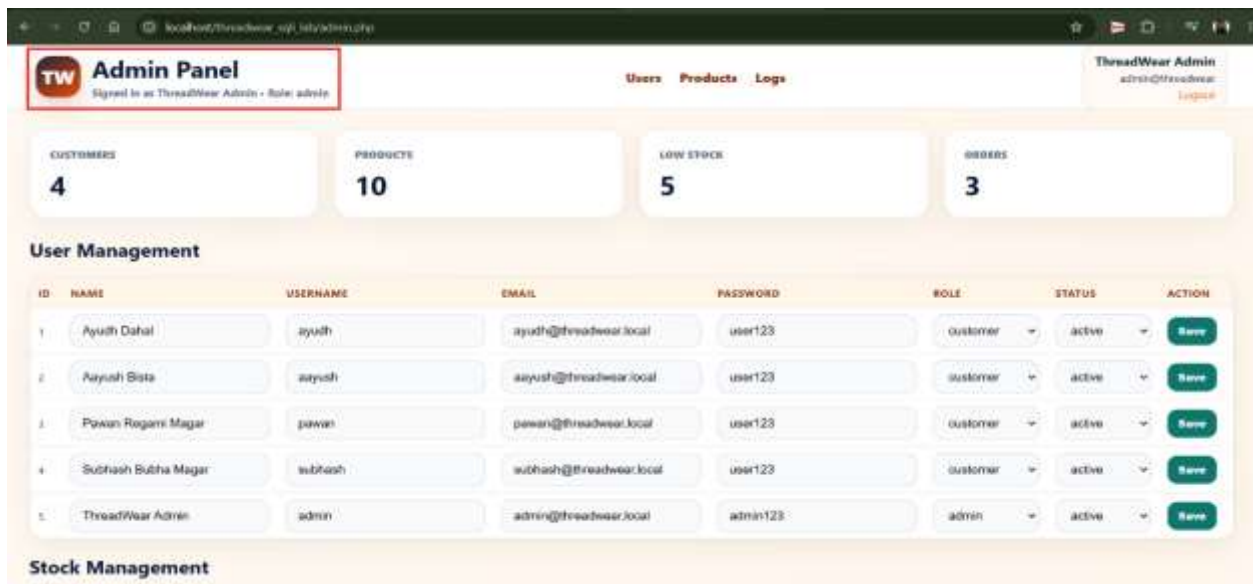


Figure 32: Successful access to the ThreadWear admin panel

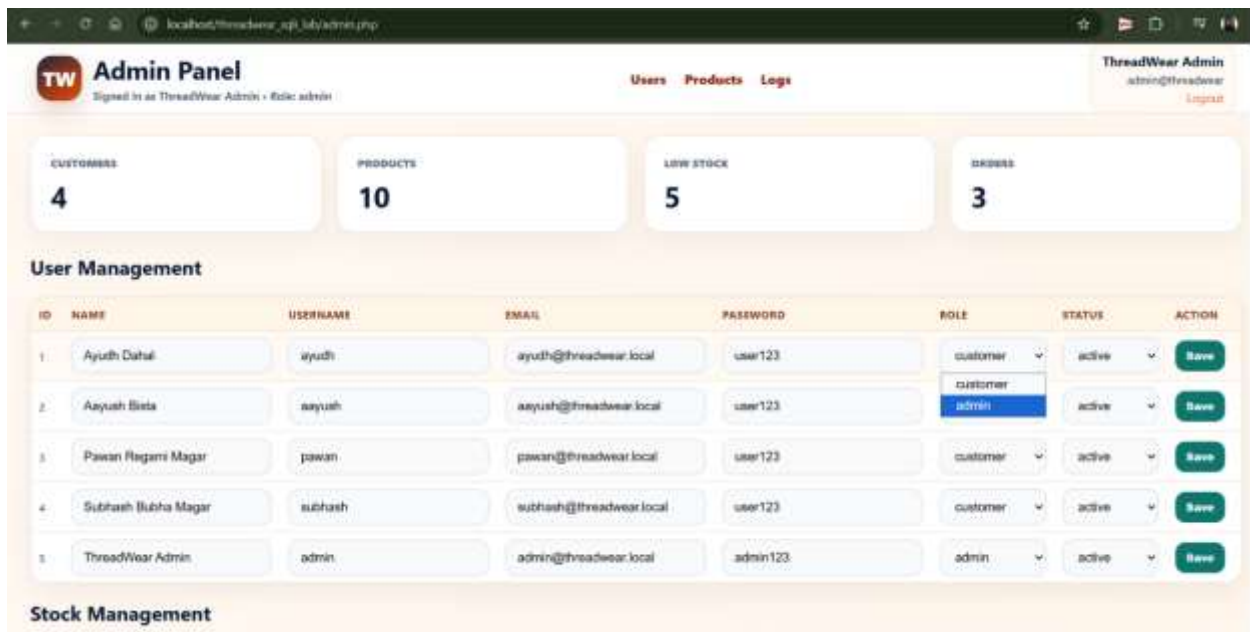


Figure 33: Role dropdown showing customer role changed to admin

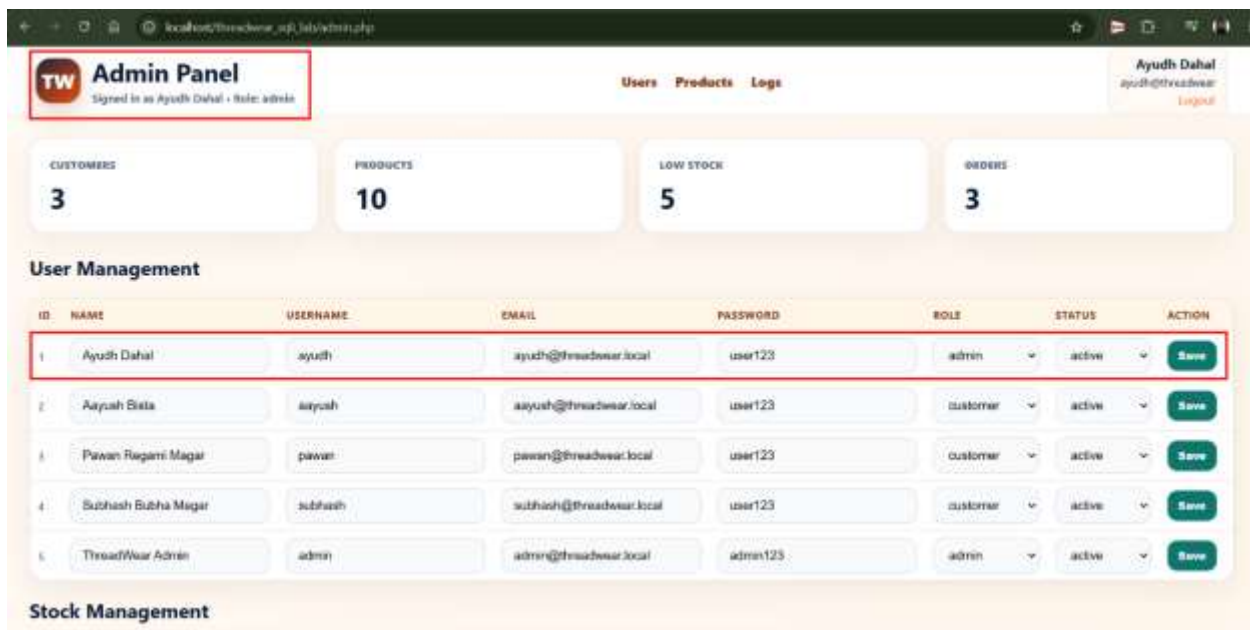


Figure 34: Ayudh Dahal account successfully promoted to admin

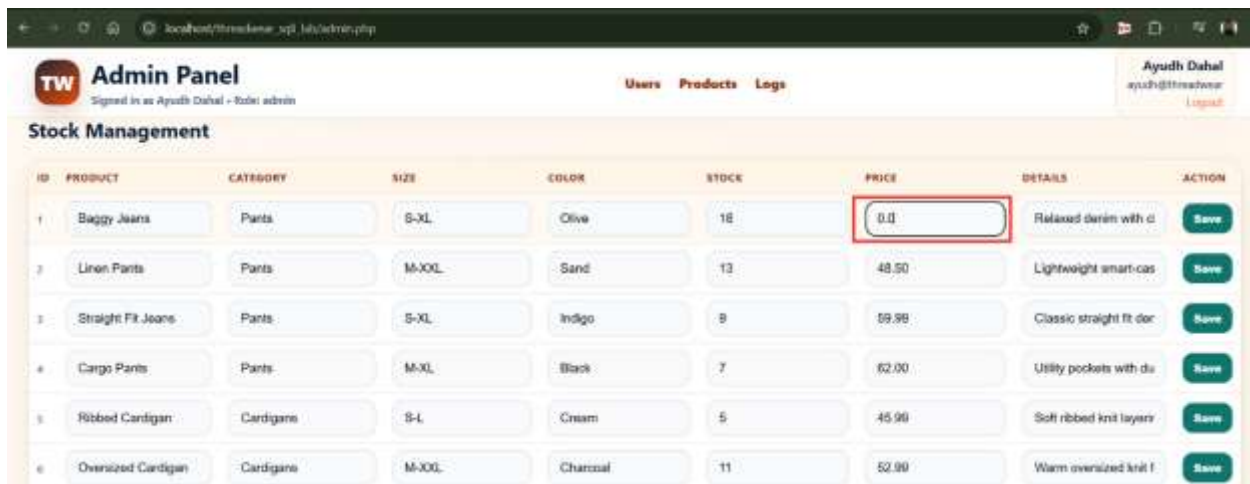


Figure 35: Product price modified to 0.00 from admin stock management.

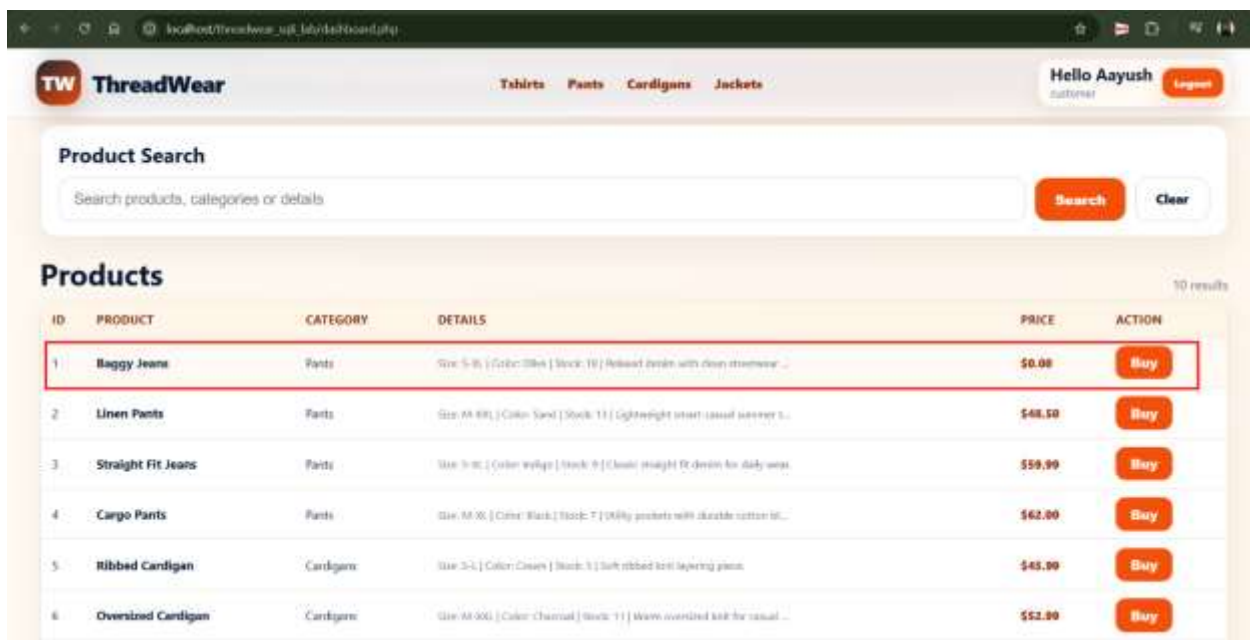


Figure 36: Dashboard showing altered product price as \$0.00

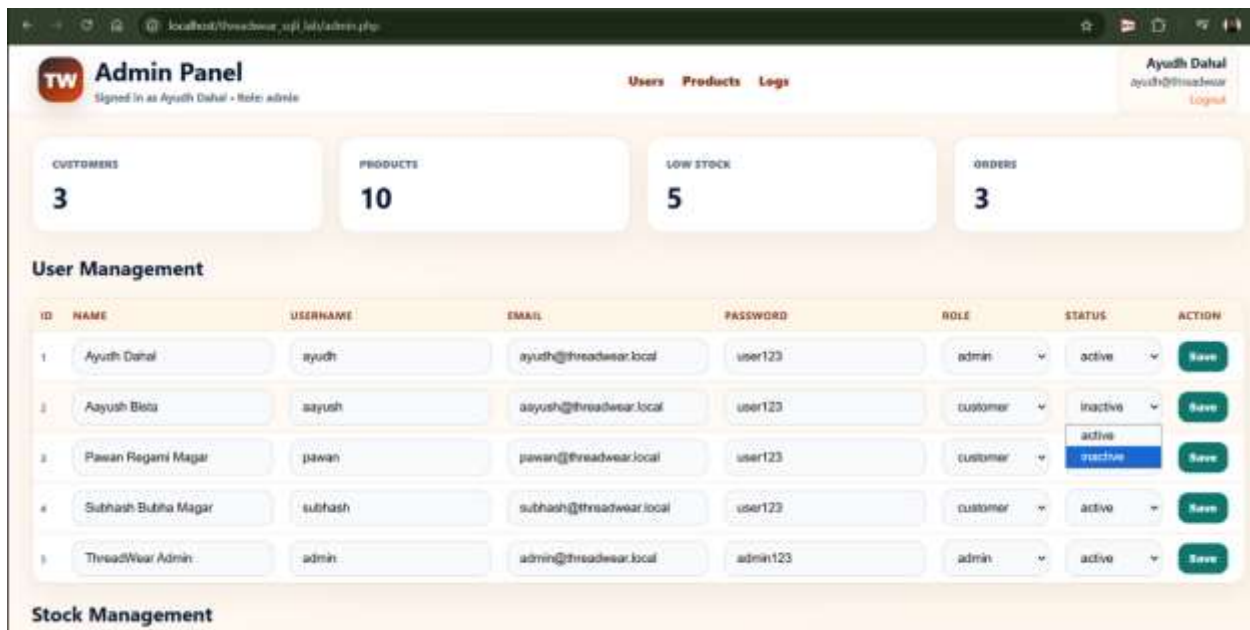


Figure 37: Disabling one user's ID

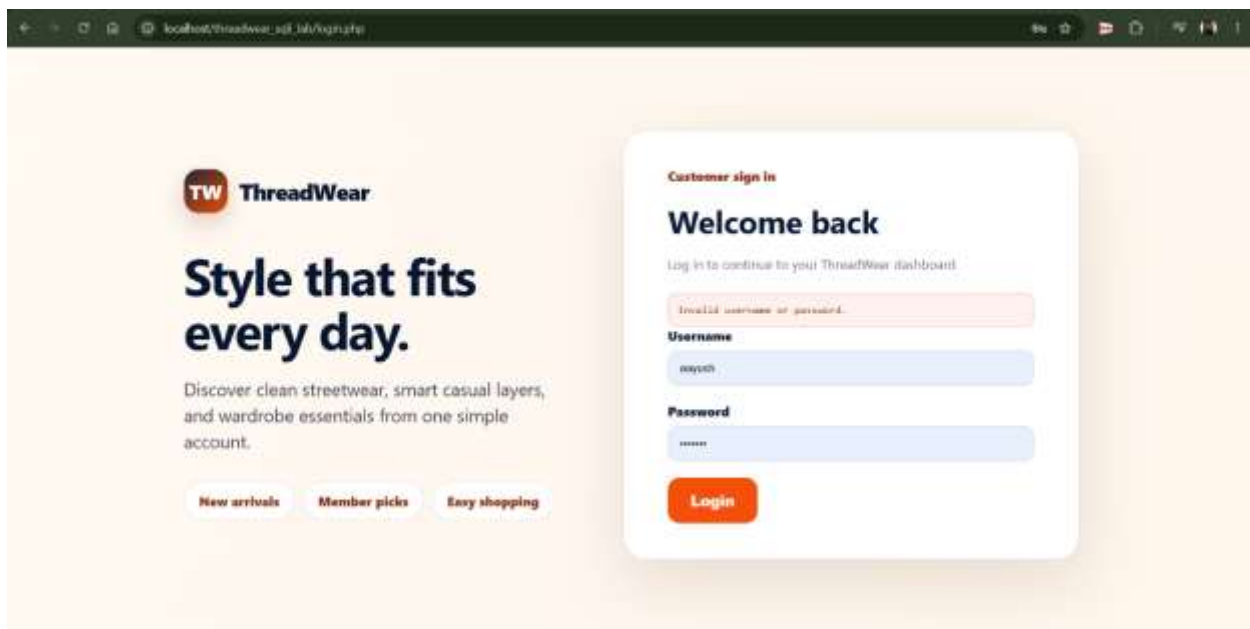


Figure 38: User cant login after

Impact Analysis and CVSS Score:

Impact Area	Evidence from Testing	Security Effect	Severity
Confidentiality	Username, passwords, and roles were extracted from the 'users' table.	Sensitive account data was exposed to an unauthorized user.	High
Privilege Escalation	A normal customer account was promoted to admin from the admin panel.	An attacker could gain administrative control over the application.	Critical
Integrity	Product price was changed to `0.00` from the stock management panel.	Business data could be modified without authorization.	High
Availability	The customer account was changed to inactive and login failed afterward.	Legitimate users could be denied access to their account.	Medium
Business Impact	Admin functions such as user, product, and account management became accessible.	The organization could suffer financial loss, service misuse, and reputation damage.	High

Figure 39: Impact Analysis

Base Score 9.6 (Critical)

Attack Vector (AV)

Attack Complexity (AC)

Privileges Required (PR)

User Interaction (UI)

Scope (S)

Confidentiality (C)

Integrity (I)

Availability (A)

Vector String - CVSS:3.1/AV:N/AC:L/PR:L/UR:N/S:C/C:H/I:H/A:N

Figure 40: CVSS Score

(For remediation process of SQL Injection and retesting of the attack after remediation, please refer to [Appendix 8.2.](#))

5. Impact of Electronic Transaction Act 2063 on Ethical Hacking

Two major points of Electronic Transaction Act 2063 are:

Section 45 – Unauthorized Access in Computer Materials

Section 45 of the Electronic Transactions Act 2063 deals with unauthorized access to computer programs, data, or information. This law is intended to protect computer materials from being accessed by a person who has no permission, or by a person who has some permission but goes beyond the access they were allowed. In simple terms, the law is violated when someone enters or reaches protected computer material without proper authorization or acts beyond the limit of the permission they were given. If Section 45 is violated, the Act provides for punishment of up to three years' imprisonment, or a fine of up to NPR 200,000, or both.

Section 46 – Damage to any Computer and Information System

Section 46 deals with the destruction, deletion, alteration, damage, or disruption of computer information or a computer system. The purpose of this law is to protect digital information and computer systems from intentional interference that may cause loss, harm, or disruption. This law is violated when a person knowingly changes, deletes, damages, or disrupts computer data or system operations without lawful authority. If Section 46 is violated, the Act provides for punishment of up to three years' imprisonment, or a fine, or both.

Figure 41: Two major laws

How this coursework followed ethical boundaries?

Answer: In this coursework, the testing was conducted only on a locally hosted ThreadWear lab application using an authorized account and agreed testing boundaries. No public system, third-party data, or external network was accessed, which keeps the practical work within ethical and legal limits while showing how SQL Injection vulnerabilities can be identified and fixed responsibly.

6. Conclusion

This coursework examined SQL Injection as a serious web application vulnerability through background research, case studies, and practical testing in a controlled ThreadWear localhost environment. The demonstration proved that the product search feature operated with insufficient input security which permitted attackers to execute SQL injection attacks that exposed complete database structures including table and column information and user account details. The research established that exposed account information would enable attackers to obtain administrator control which would allow them to escalate their privileges and modify product prices and disrupt user accounts.

The implementation of prepared statements with parameter binding replaced the direct SQL query concatenation to fix the identified security weakness. The testing process showed that the same test inputs did not result in SQL errors or database exposure or unauthorized access to user information. The application requires future development to implement password hashing and database accounts with minimum required access rights and enhanced input validation and security logging systems and expanded testing using Burp Suite and OWASP ZAP tools.

7. References

- ACTE. (2016). *Ethical Hacking*. Retrieved from ACTE: <https://www.acte.in/guide-to-ethical-hacking-from-scratch>
- Akamai. (2025). *Web application attacks and API attacks* | Akamai. Akamai.
- CWE. (2004). *CWE-89 : Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')*. Retrieved from Common Weakness Enumeration: <https://cwe.mitre.org/data/definitions/89.html>
- Halfond, W. G., Viegas, J., & Orso, A. (2006). *A Classification of SQL Injection Attacks*. Georgia Institute of Technology.
- Heydari, M., & Tajpour, A. (2010). *SQL injection detection and prevention tools assessment*. Retrieved from Researchgate: Atefeh Tajpour
- ICO. (2015). *TalkTalk cyber attack – how the ICO’s investigation unfolded*. Retrieved from TalkTalk cyber attack – how the ICO’s investigation unfolded: <https://ico.org.uk/about-the-ico/media-centre/talktalk-cyber-attack-how-the-ico-investigation-unfolded/>
- INCIBE - CERT. (2020). *Freepik Company suffers credential theft due to SQL injection*. Retrieved from Freepik Company suffers credential theft due to SQL injection: <https://www.incibe.es/en/incibe-cert/publications/cybersecurity-highlights/freepik-company-suffers-credential-theft-due-sql-injection>
- INCIBE -CERT. (2025). *Freepik Company suffers credential theft due to SQL injection*. Retrieved from Freepik Company suffers credential theft due to SQL injection: <https://www.incibe.es/en/incibe-cert/publications/cybersecurity-highlights/freepik-company-suffers-credential-theft-due-sql-injection>
- NIST. (2011). *Information System Security*. California, USA: National Institute of Standards and Technology .
- OWASP. (2016). *SQL Injection*. Retrieved from SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection

OWASP. (2025). *OWASP Top 10:2025*. Retrieved from OWASP:
<https://owasp.org/Top10/2025/>

OWASP. (n.d.). *OWASP Cheat Sheet*. Retrieved from
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Portswigger. (2020). *What is SQL Injection?* Retrieved from Portswigger:
<https://portswigger.net/web-security/sql-injection>

Riley, D. (2020). *8.3M records of Freepik and Flaticon users stolen in SQL injection attack*. Retrieved from <https://siliconangle.com/2020/08/24/8-3m-records-freepik-flaticon-users-stolen-sql-injection-attack/>

Verizon. (2024). *2024 Data Breach Investigations Report | Verizon*. Verizon.

Verizon. (2024). *2024 Data Breach Investigations Report | Verizon*. Verizon.

VMWare. (2024). *SQL Injection Attack Definition*. Retrieved from VMWare:
<https://www.vmware.com/topics/sql-injection-attack>

8. Appendix

8.1. Background

8.1.1. Types of SQL Injection

SQL Injection attacks use various attack methods which depend on the vulnerable application processing of attacker input and the way it returns results to web browsers. The process of SQL Injection testing involves verifying whether user input gets inserted into database queries through methods which enable attackers to modify the original SQL query structure (OWASP, 2016). The ThreadWear lab identified the product search field as their main security weakness because the search input showed up in a database-based product result table.

Three types of SQL Injection attacks are listed below:

8.1.1.1. In-band SQL Injection

Two types of In-band SQL Injection are:

- **Error-based SQL Injection**

It occurs when an application displays database errors after malicious input is supplied. The system generates errors which provide essential information about SQL syntax errors and column count discrepancies and database functions and table structures. The application generates an error message which shows the selected column does not exist when users enter an incorrect ORDER BY value. According to OWASP error messages enable attackers to determine the actual SQL query which they can use for further exploitation (OWASP, 2016) .

- **Union-based SQL Injection**

It uses the SQL UNION SELECT operator to combine the original query with another query controlled by the attacker. The technique operates best when users see the results from the query which contains security weaknesses. The ThreadWear lab team used UNION-based payloads to show database information which included database names and table names and column names and user account records within the product result table. The UNION operator appears as a standard SQL Injection attack method which OWASP describes for combining database output results (OWASP, 2016).

8.1.1.2. Out-of-band SQL Injection

The out-of-band SQL Injection technique is one in which the attacker is unable to obtain the result through the conventional web application. The database is made to transmit information or signals through a channel other than the web application like DNS or HTTP requests to an external server owned by the hacker. This approach is commonly used in situations where the error-based, union-based, or boolean-based approaches cannot be applied because the application fails to provide useful output. Out-of-band SQL injections require that the database server can perform network operations. The out-of-band SQL injection method could not be demonstrated in the ThreadWear lab due to the limitations of the local environment and the primary goal of demonstrating SQL injection using error-based, boolean-based, and union-based techniques.

8.1.1.4. Blind SQL Injection

Two types of Blind SQL Injection are:

- **Boolean-based blind SQL Injection**

It is used when the application does not directly display database information but responds differently to true and false conditions. The payload ' AND 1=1-- - shows standard output but the payload ' AND 1=2-- - fails to display any products. The attacker uses different system responses to determine which conditions become true or false. OWASP classifies this as an inferential or blind technique because the attacker reconstructs information by observing application behaviour rather than receiving direct output (OWASP, 2016).

- **Time-based blind SQL Injection**

It is an inferential method which enables attackers to create intentional database response delays for information extraction. The function SLEEP (5) serves as a payload method which verifies SQL injection execution when the website shows no output. The website response time makes it obvious that the injected commands have been processed by the database system. (OWASP, 2016).

(Back to [2. Background](#))

8.2. Remediation and Retesting

The SQL injection issue was fixed in dashboard.php, through a modification which stopped the search parameter q from being directly added to database queries. The insecure database query was replaced with a prepared statement which uses prepare() and bind_param() and execute() for execution.



```

1  <?php
2  session_start();
3  if (!isset($_SESSION['user_id'])) { header('location: login.php'); exit; }
4  if ($_SESSION['role'] !== 'admin') { header('location: admin.php'); exit; }
5  require_once 'db.php';
6  $q = $_GET['q'] ?? '';
7  $displayError = '';
8  $rows = [];
9  $productCategories = ['Pants', 'Cardigans', 'Tshirts', 'Jackets'];
10 mysql_report(MYSQL_REPORT_OFF);
11 if ($q !== '') {
12     $sql = "SELECT id, name, category, CONCAT('Size: ', size, ' | Color: ', color, ' | Stock: ', stock, ' | ', details) AS details, price FROM products WHERE id LIKE '$q%'";
13 } else {
14     $sql = "SELECT id, name, category, CONCAT('Size: ', size, ' | Color: ', color, ' | Stock: ', stock, ' | ', details) AS details, price FROM products WHERE name LIKE '%$q%'";
15 }
16 $res = $conn->query($sql);
17 if ($res) { while($row=$res->fetch_assoc()) { $rows[] = $row; } }
18 else { $displayError = $conn->error; }
19
20

```

Figure 42: Vulnerable query before remediation



```

1  <?php
2  session_start();
3  if (!isset($_SESSION['user_id'])) { header('location: login.php'); exit; }
4  if ($_SESSION['role'] !== 'admin') { header('location: admin.php'); exit; }
5  require_once 'db.php';
6  $q = $_GET['q'] ?? '';
7  $displayError = '';
8  $rows = [];
9  $productCategories = ['Pants', 'Cardigans', 'Tshirts', 'Jackets'];
10 mysql_report(MYSQL_REPORT_OFF);
11 if ($q !== '') {
12     $stmt = $conn->prepare("SELECT id, name, category, CONCAT('Size: ', size, ' | Color: ', color, ' | Stock: ', stock, ' | ', details) AS details, price FROM products WHERE id LIKE ?");
13 } else {
14     $search = '%' . $q . '%';
15     $stmt = $conn->prepare("SELECT id, name, category, CONCAT('Size: ', size, ' | Color: ', color, ' | Stock: ', stock, ' | ', details) AS details, price FROM products WHERE name LIKE ?");
16     $stmt->bind_param('s', $search, $search, $search);
17 }
18 if ($stmt && $stmt->execute()) {
19     $res = $stmt->get_result();
20     while($row = $res->fetch_assoc()) {
21         $rows[] = $row;
22     }
23 } else {
24     $displayError = "Search could not be completed.";
25 }
26
27

```

Figure 43: Secure query after remediation

After the fix, the same payloads were tested again. The single quote payload did not show a SQL error, the ' OR '1'='1 payload did not return all products, and the **UNION SELECT** payload did not reveal user data. The product search field demonstrated successful protection against SQL injection attacks during testing.



Figure 44: Single quote retest



Figure 45: Tautology payload retest



Figure 46: UNION SELECT retest

(Back to [Impact Analysis and CVSS Score](#))